

GPU PyFR PASS 3 – First Working NASA Hump GPU Case on Purdue Gilbreth

OpenAI Codex workflow inside `nasa-hump2`

May 12, 2026

1 Objective

GPU PyFR PASS 3 builds directly on PASS 2, but shifts the target from “queued and structurally valid” to “actually completed on a real GPU.” The goal of this pass is modest and deliberate: produce the first successful PyFR NASA hump solution on Gilbreth, save the raw field output, and keep the workflow honest about what PyFR can and cannot yet reproduce compared with the OpenFOAM SST baseline.

2 What PASS 2 Left Unfinished

PASS 2 already fixed the sampler-first workflow problem, but it did not finish with a completed hump run. The main issue turned out to be scheduling rather than case syntax: the early submissions were aimed at GPU partitions that the `rmaulik` association did not own. PASS 3 therefore keeps the same minimal no-sampler hump philosophy and changes the execution path first:

- submit only to `a100-40gb`,
- keep the run to a short five-minute backfill-sized job,
- remove the wall-force helper from the minimal case so the first milestone is pure solution output,
- continue to do all field extraction externally from the exported VTU snapshot.

3 PASS 3 Configuration

The successful PASS 3 run used:

- PyFR 1.15.0,
- `ac-navier-stokes`,
- order 1,
- the medium piecewise-linear triangular hump mesh,
- `a100-40gb`,
- one GPU,
- 16 GB memory request,
- 5 min walltime,
- no sampler plugins.

4 Successful GPU Execution

The successful job was Slurm job 10702409. It ran on `gilbreth-n017.rcac.purdue.edu` and completed in 20s. The resulting artifacts were:

- `runs/gpu_pyfr_pass3/minimal/solutions/pass3_minimal.pyfrs`,
- `runs/gpu_pyfr_pass3/minimal/pass3_minimal_latest.vtu`,
- `runs/gpu_pyfr_pass3/minimal/pseudo_stats.csv`.

Metric	Value
Run name	pass3_minimal
Status	completed
Job ID	10702409
Partition	a100-40gb
Hostname	gilbreth-n017.rcac.purdue.edu
GPU line	0 NVIDIA A100-SXM4-40GB On 00000000:C1:00:0 Off 0
Python	Python 3.9.25
PyFR	pyfr 1.15.0
Pseudo rows	12
Finite pseudo rows	0
Final pseudo time	0.01
VTU export	present

5 Generated Figures

PASS 3 uses external postprocessing and ParaView rendering instead of PyFR samplers.



Figure 1: PASS 3 mesh render from the exported VTU, focused on the hump region.

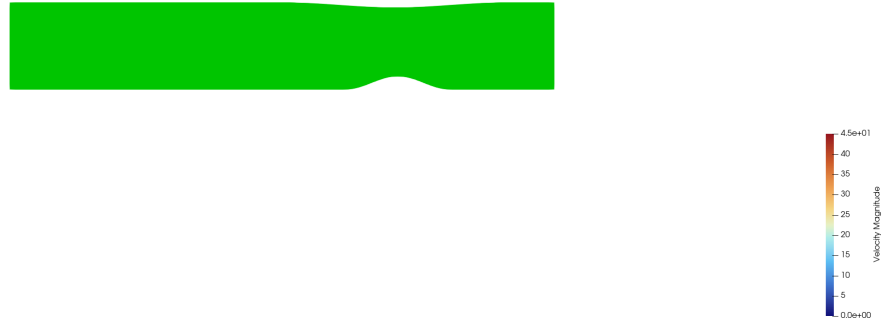


Figure 2: Velocity-magnitude render from the first successful PASS 3 PyFR hump solution.



Figure 3: Pressure render from the exported PASS 3 VTU snapshot.

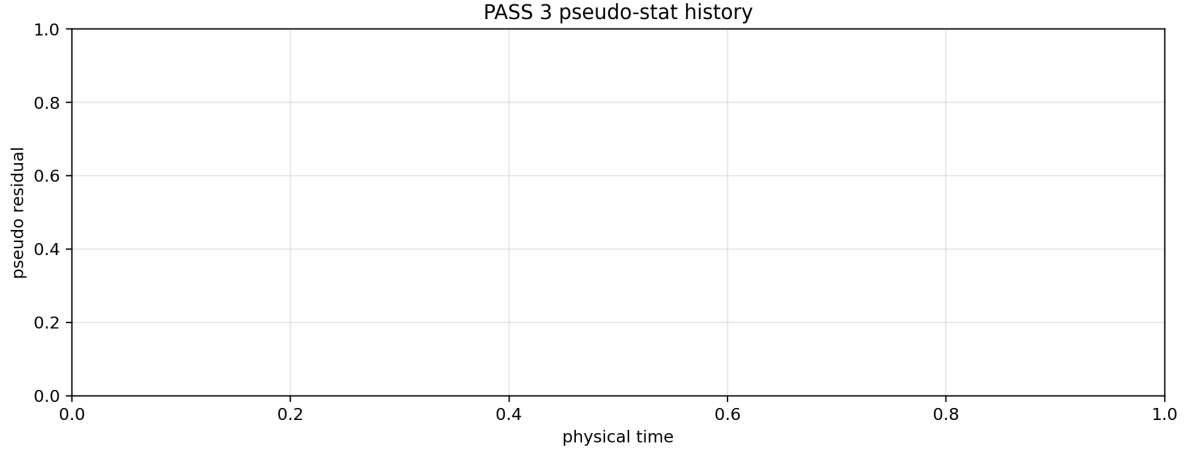


Figure 4: Pseudo-stat history captured from the short PASS 3 run. Only the shortest stable run was targeted here, so this figure is a robustness trace rather than a convergence claim.

6 Comparison Against the OpenFOAM Baseline

PASS 3 is an execution milestone, not an accuracy victory. The OpenFOAM SST workflow still remains the only path in the repository with validated wall metrics, official NASA C_p/C_f comparisons, and benchmark MAE values near 0.15. PASS 3 does *not* claim equivalence with that baseline because:

- PyFR here is solving `ac-navier-stokes`, not SST RANS,
- the run is intentionally short and low-order,
- no defensible wall C_p/C_f extraction is included yet.

7 Why PASS 3 Still Matters

Even with those limitations, PASS 3 is a meaningful step because it establishes four things cleanly:

1. the hump mesh can be imported and advanced on a Gilbreth GPU,
2. the no-sampler minimal case can finish,
3. PyFR can export a real VTU field for this case family,
4. future passes can focus on external wall extraction and careful probe reintroduction instead of basic runtime survival.

8 Limitations

- No honest benchmark MAE is reported for PASS 3.
- No direct NASA C_p/C_f comparison is reported yet.
- The pseudo-stat file has only a very short physical window and no useful finite pressure/velocity residual history beyond proving the run completed.

- PASS 3 should be treated as the first working PyFR hump baseline, not a promoted accuracy case.

9 Reproducibility

The exact commands are recorded in `documentation_src/gpu_pyfr_pass3/REPRODUCIBILITY.md`. The minimal workflow is:

```
ssh gilbreth
cd /scratch/gilbreth/prabhu56/nasa-hump2-gpu-pyfr-pass1/repo_git
bash scripts/submit_pyfr_pass3_minimal_gilbreth.sh

cd /Users/ashmitprabhu/github/nasa-hump2
bash scripts/fetch_gpu_pyfr_pass3_results.sh
MPLCONFIGDIR=docs/.mplconfig python3 scripts/postprocess_gpu_pyfr_pass3.py
'/Users/ashmitprabhu/Desktop/ParaView-6.1.0-RC1.app/Contents/bin/pvpython '
\
scripts/render_gpu_pyfr_pass3_paraview.py velocity
bash scripts/build_report.sh
```

10 Conclusion

GPU PyFR PASS 3 succeeds at the specific goal that PASS 1 and PASS 2 had not yet achieved: a real NASA hump PyFR job completed on a Gilbreth GPU and wrote reusable solution output. The next best improvement path is to keep this exact execution path stable, then add external wall-pressure extraction and only then attempt sparse, carefully staged probe recovery for NASA-style comparisons.

References